

The **fastest** way to build a **fast** trading stack.

Market data, messaging, storage, backtesting, execution —
one ultra-low-latency Rust graph.



BATTERIES INCLUDED · OPEN SOURCE

Every backend speaks a different **language**.

Sockets. Brokers. Databases. Each with its own client, its own threading model, its own idea of time.

Market data feeds

UDP multicast, shared memory, vendor SDKs

Message buses

Kafka, Fluvio, ZeroMQ — each a bespoke consumer loop

Databases

request/response, batch, time-series quirks

Execution venues

FIX sessions, sequence numbers, heartbeats

Wiring them into one low-latency graph leaks **boilerplate** — and **latency**.

A graph engine for **real-time Rust.**

You declare a directed graph (DAG) of transformations. wingfoil runs it — breadth-first, in dependency order, on every tick.

DAG engine

Nodes declare upstreams; the engine schedules them. ~20 ns of overhead per node cycle.

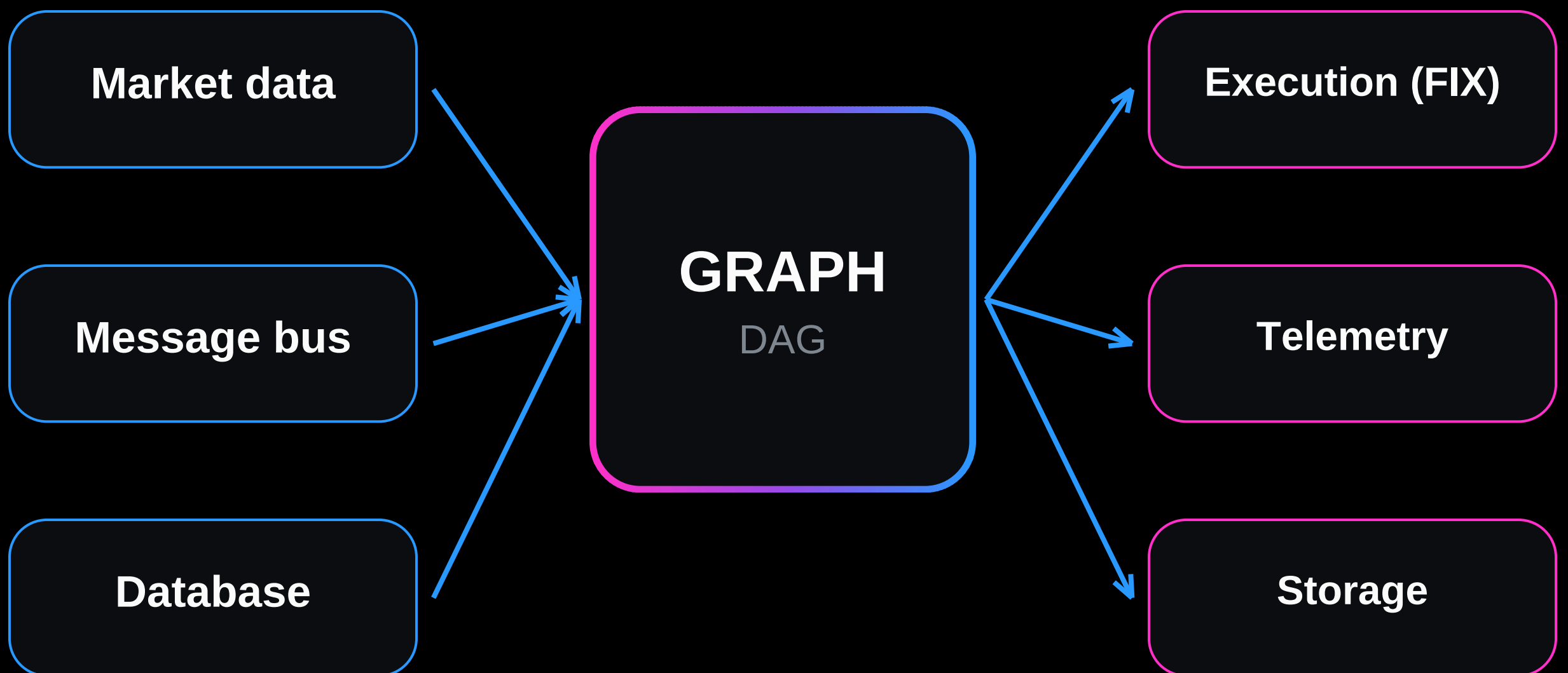
Adapters at the edges

I/O lives at the boundary of the graph. The core stays pure data-flow.

Async + multi-threaded

Tokio-backed async sources and sinks, plus dedicated OS threads where latency demands it.

Adapters are the edges of the **graph**.



Sources → [graph] → sinks.

Swap an adapter; the graph doesn't change.

A whole trading stack, included.



MARKET DATA IN

Aeron · iceoryx2 · ZeroMQ



MESSAGING

Kafka · Fluvio



STORAGE + REPLAY

kdb+ · your DB?



EXECUTION OUT

FIX



OBSERVABILITY

OpenTelemetry · Prometheus

+ CSV · etcd · WebSocket — or build your own.

Zero-copy **shared memory**

iceoryx2 hands you bytes straight out of shared memory — no serialization, no copy on the hot path.



```
// zero-copy IPC — no serialization on the hot path
let trades = iceoryx2_sub::<Trade>("md/trades");

trades
    .collapse()
    .for_each(|t, _| route(t))
    .run(RunMode::RealTime, RunFor::Forever)?;
```

Payloads are `#[repr(C)]` — nanoseconds, not microseconds.

One consumer loop

Subscribe to a topic and it becomes a graph source. Bursts of messages flow downstream like any other stream.

● ● ●

```
let conn = KafkaConnection::new("localhost:9092");

kafka_sub(conn, "orders", "risk-engine")
    .collapse()
    .map(|ev| decode(ev.value))
    .run(RunMode::RealTime, RunFor::Forever)?;
```

Same fluent API as **every** other source.

History as a stream

kdb_read pulls history in time-slices via a closure you write. Time lives on the graph, not in your records.



```
let conn = KdbConnection::new("localhost", 5000);

kdb_read::<Trade, _>(conn, hour,
  |(t0, t1), date, _| format!(
    "select from trades where \
      date=d+{date}, time>={t0}, time<{t1}"))
  .collapse().map(|t| t.price)
  .run(HistoricalFrom(start), Duration(day));
```

Same graph. History or real-time.

REPLAY = BACKTESTING

Backtest and production are the **same graph**.

Flip one argument. The adapters, the nodes, the wiring — all identical. Only the clock changes.

BACKTEST



```
// replay history through the graph
graph.run(RunMode::HistoricalFrom(start),
          RunFor::Duration(day));
```

PRODUCTION



```
// same adapters, same graph, wall clock
graph.run(RunMode::RealTime,
          RunFor::Forever);
```

Test on history. **Deploy unchanged.**

Built for the hot path.

~20 ns

engine overhead per node, per cycle
(10×10 DAG, all nodes ticking, 3.8 GHz)

DAG execution

Breadth-first, dependency-ordered. Cached upstream indices — no per-tick map lookups.

Zero-copy adapters

Shared-memory IPC and `#[repr(C)]` payloads skip serialization on the critical path.

Async + multi-core

Tokio sources, dedicated OS threads, and spin modes where every microsecond counts.

Build your favorite DB adapter.

kdb+ is in. Postgres, ClickHouse, DuckDB, Redis aren't — yet.
There's a skill + template to make it a weekend project.

1

Star the repo

Two seconds. Helps more devs find it.

2

Grab a good-first-issue

Scoped adapter issues, ready to claim.

3

Contributor mentorship

We pair with you through your first PR.

github.com/wingfoil-io/wingfoil